

How Hardware Gets Hacked (Part 10)

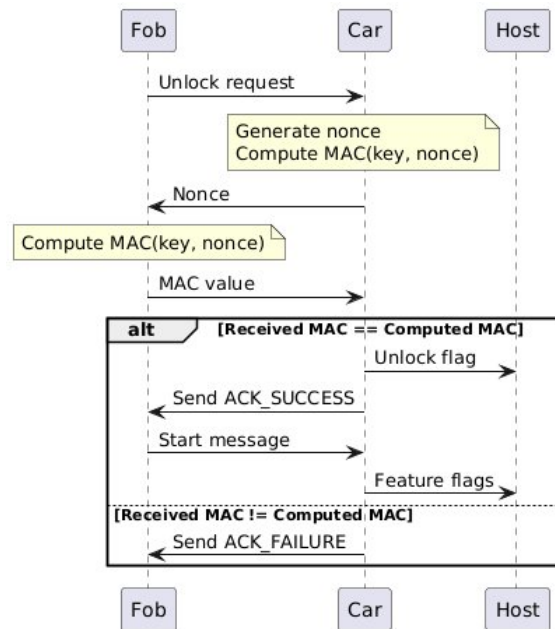
The “Birthday Problem” Attack

by Nathan Jones

Introduction	1
The “Birthday Problem”	2
Attack #8: The “Birthday Problem” attack	6
Defense #8: Widen the nonces	7
Other defenses	8
The problem with “oracles”	10
Updated threat model	12
Conclusion	14

Introduction

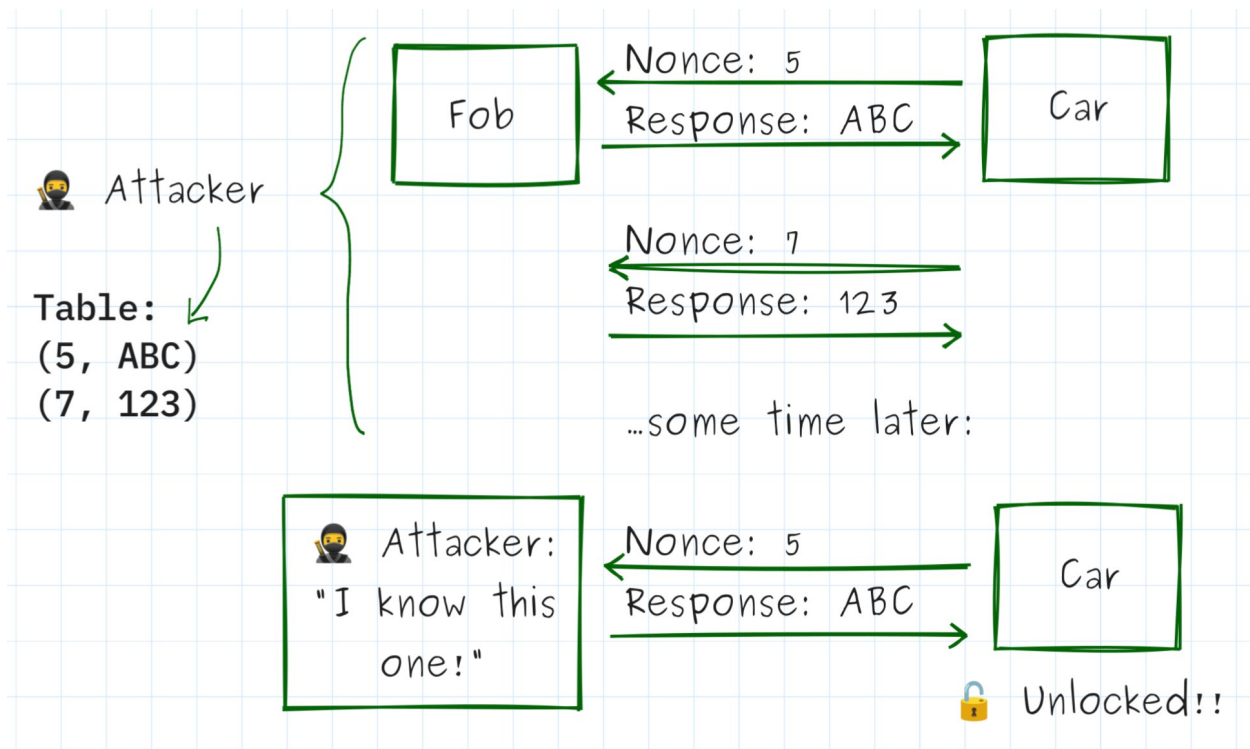
When we [last left the unlocking process](#), things seemed pretty secure: the car generates a totally random nonce (“number used only once”) which the attacker couldn’t possibly have guessed ahead of time. The fob then has to provide the proper MAC value of this nonce, computed using the `unlock_key`, which is unknown to an attacker. What could go wrong?



Ah, famous last words. In this article, we'll discover *yet another* type of replay attack based on something called the “birthday problem” and the fact that even PRNGs repeat values WAY more often than you'd think they would.

The “Birthday Problem”

Let's take a closer look at the phrase “totally random nonce.” In reality, this isn't quite true. Sure, an attacker can't guess what nonce values are coming next. But the nonce itself has a predetermined size, so there are only so many nonces that could be generated before our PRNG has to emit a duplicate. If our car repeats a nonce more than once and an attacker happened to save the fob's MAC response for that first nonce, then we're back in replay territory!



All an attacker has to do is collect enough (nonce, response) pairs for the car to repeat a nonce and then BOOM: they send back the recorded response and they can get the car to unlock.



Two examples of an attacker who could collect these (nonce, response) pairs are

1. a valet who is holding a person's key fob while they're eating dinner or
2. an attacker who places a listening device outside a person's house to capture unlocks every time that person gets into their car over the course of a year.

In both cases, the attacker records enough valid unlock responses that when they return to the car and try to unlock it themselves, they have a good chance of knowing the correct MAC response without needing to know the unlock key.

“But c’mon! Nonces are currently 32 bits wide, giving over *4.3 billion* possible values. How likely is it, really, for an attacker to find a repeated nonce?”

Let’s find out! First, let’s assume that an attacker can trigger an unlock and store the (nonce, response) pair every 3.4 ms¹. This means that if an attacker had control of a fob for a mere 30 minutes (like a valet might) they could potentially trigger 530,973 unlocks. We want to find out how long would it take that attacker to later break into that car by requesting unlocks until the car challenged them with one of the 530,973 nonces they recorded earlier.

The general statement for this problem is “Given a set of T values from a distribution of size N, what’s the probability that in M random draws from that set at least one of them matches a value from T?” It’s known as the “birthday problem” since another example of this exact same question is “Assume you have a group of T people with different birthdays and you ask M random strangers for *their* birthdays. What are the chances that at least one of those M people shares a birthday with someone in the group of T people?”² (in this case, “dates in a year” forms the pool of “N” values, N being 365).

The formula for this probability is $P(M) = 1 - \left(1 - \frac{T}{N}\right)^M$, which can be approximated as $P(M) \approx 1 - e^{-TM/N}$. Weirdly, M doesn’t need to be that big to have a really good chance of finding a match! If you took a group of 20 people with different birthdays, it would only take about *12 random strangers* to have a 50/50 chance of finding someone who shared a birthday with a person from the original group!

¹ A full unlock is an unlock message (3 bytes), followed by a nonce message (6 bytes), a response message (10 bytes), an ACK message (3 bytes), and a start message (17 bytes). At 115200 baud, and assuming no processing delays between messages, this entire series of messages only takes 3.39 ms.

² This is technically a form of the “birthday problem” called a “two-set cross-collision”, the two sets being T and M. A simpler formulation (the one you’re likely to find described if you search for “birthday problem”) is a “single-set cross-collision”, in which we ask “given a set of T values taken from a distribution of size N, what’s the probability that at least two of those T values are the same?” In birthday terms, this is like asking “Given a group of T people, what are the chances that two of them share a birthday?”

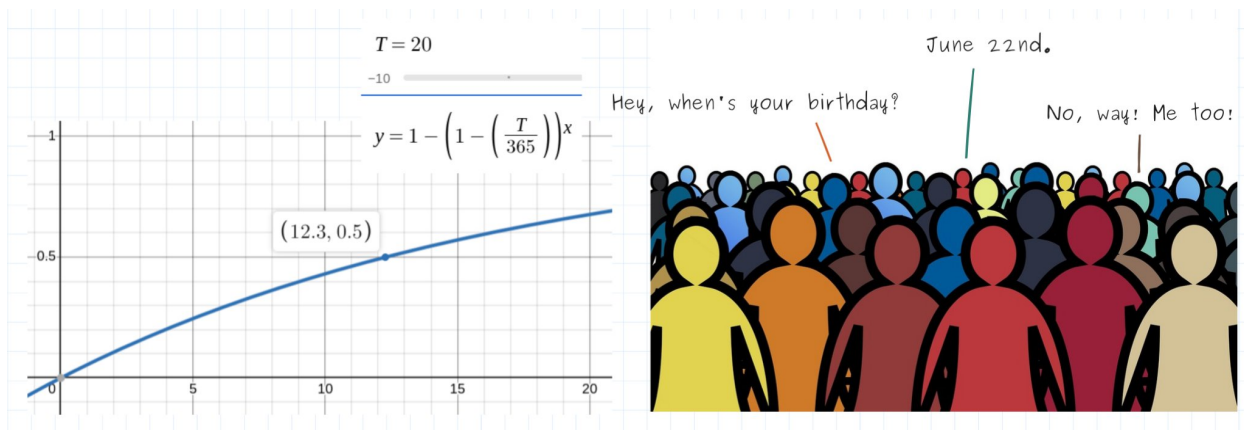


Figure 1: In a group of 20 people, it would only take about 12 random strangers to have a 50% chance of finding someone who shared a birthday with one of the people from the group. Image from <https://picryl.com/media/crowd-human-silhouettes-6333fd>.

/*****[! Test it out for yourself!]***/

Generate a set of 20 random values from 1 to 365. How long does it *feel* like it will take to get a repeated value (regardless of what the math says)?

Now generate random values until you get one that matches one of the original 20 values. How many random numbers did you need to generate to find that match? What was the probability of finding a match when you did?

*****/

In the case of our key fob, $T = 530,973$ and $N = 2^{32} = 4,294,967,296$. Let's say the attacker had another 30 minutes to try to get into the car using that table, during which time they could request another 942,408 unlocks (our value for M), since each failed unlock attempt would only take 1.91 ms^3 .

/*****[? What's the probability?]***/

So, what's the probability that our attacker finds a repeated nonce within that window? Calculate it before looking at my answer below!

*****/

Let's see, $P(M) = 1 - \left(1 - \frac{T}{N}\right)^M = 1 - \left(1 - \frac{530973}{4294967296}\right)^{942408} = 1$.

Woah, it's basically guaranteed?! I'm afraid so. In fact, it gets even worse. We can rearrange the "birthday problem" formula to calculate approximately how many unlocks (M) we'd need to have a 50% chance of a repeated value, and that formula is $M = \frac{\log(1-P)}{\log(1-T/N)}$.

³ A failed unlock attempt includes everything from a normal unlock transaction minus a start message. See footnote 1 for how that time could be calculated.

For our 4.3 billion nonces and 530,973 table size, an attacker would only need to trigger about **5,606 unlocks** before they had a 50% chance of seeing a repeated value, which they could do with our system in as little as **11 seconds**.

/*****[? What about the second scenario?]***/

In the second scenario above, an attacker placed a listening device outside a person's house to capture each of their unlocks over the course of a year.

What's a reasonable assumption for how many unlocks that attacker might capture in that year (i.e. the size of T)?

How long would it take an attacker who had that table to unlock a car using this attack? (Answerⁱⁱ)

*****/

Attack #8: The “Birthday Problem” attack

This is exactly the threat model described by **Car #2** in the competition, in which an attacker (such as a valet or anyone who “borrows” your keys) only has temporary access to your fob.

 = Full Access  = Temporary Access	Car	Paired Fob	Unpaired Fob	Logic Analyzer Capture of Unlock	Pairing PIN	Packaged Feature 1	Packaged Feature 2
Car 2 Temporary Fob Access							

The security test I wrote performs exactly the attack just described, using `getBoardMsgLog` to simulate eavesdropping on each unlock transaction. It collects a set number of (nonce, response) pairs (configurable on the command-line with `--oracle-table-size`; default: 65535) then triggers unlocks until it finds a nonce that matches one from the table (or until `--oracle-max-iter` is reached; default: 0, meaning run indefinitely).

In simulation, this test only takes an astonishing **40 seconds** to fail.

```

> pytest testing/test_security.py::TestComplexReplayAttacks::test_birthday_bound_attack_full --run-birthday-bound-attack-full
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2826/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 1 item

testing/test_security.py F [100%]

===== FAILURES =====
testing/test_security.py:309: in test_birthday_bound_attack_full
    TestComplexReplayAttacks.test_birthday_bound_attack_full
    assert collision_nonce is None, (
E   AssertionError: Nonce 9192da91 repeated one already in a 65533-entry table after 181982 total unlocks (36447 monitoring unlocks) - an attacker who recorded that table during a temporary fob-access window could have replayed the old response and unlocked the car without the fob
E   assert b'\x91\x92\xda\x91' is None
----- Captured stdout call -----
Built a table of 65533 (nonce -> response) pairs from 65535 unlocks.
===== short test summary info =====
FAILED testing/test_security.py::TestComplexReplayAttacks::test_birthday_bound_attack_full - AssertionError: Nonce 9192da91 repeated one already in a 65533-entry table after 181982 total unlocks (36447 monitoring unlocks) - an attacker who recorded that table during a temporary fob-access...
1 failed in 39.75s

```

Defense #8: Widen the nonces

So, *apparently* 4.3 billion possible nonce values isn't enough. (Perhaps you were thinking exactly that way back in [Part 7](#) when I made nonces only 32 bits long.) The easy fix is to make nonces longer, up to the 16 bytes that are generated by our PRNG each time we request a new random number.

```
// messages.h
#define NONCE_SIZE 16 // nonce is 16 bytes
```

If an attacker now attempts to unlock our car using their table of 530,973 stored (nonce, response) pairs, it will take them over 4.16×10^{17} centuries (over 30 million times the age of our universe) to even have a 50% chance of success! (Note that the time per attempt went up from 1.91 ms to 2.95 ms, since nonce messages are now 18 bytes instead of 6.)

$$Time = M \text{ attempts} \cdot 2.95 \frac{ms}{attempt} = \frac{\log(1 - 0.5)}{\log\left(1 - \frac{530973}{2^{128}}\right)} \cdot 2.95 = 4.44 \times 10^{32} \cdot 2.95 = 1.31 \times 10^{30} ms$$

Our current test would try to corroborate this by running for at least 4.16×10^{17} centuries before finding a nonce collision. This is, clearly, terribly unreasonable. Although we can, and should, let this test run for long periods of time to substantiate that the attack is infeasible (I've run the actual "birthday problem" attack for longer than 8 hours with no repeated nonce values), it would be nice if we had a faster test to still check that we've implemented a reasonable defense against the attack. To that end, I've added another security test that just checks the length of the nonce values and calculates the probability of the attack using the equations above. This does *not* simulate the "birthday problem" attack, but it's better than nothing for a quick security test. This test passes.

```
> pytest testing/test_security.py -k test_birthday_bound_attack_quick_check
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 15 items / 14 deselected / 1 selected

testing/test_security.py .

===== 1 passed, 14 deselected in 0.94s =====
```

Lowering the nonce length back to 4 bytes and re-running the test also confirms that our earlier design would have failed this test.

```
> pytest testing/test_security.py -k test_birthday_bound_attack_quick_check
===== test session starts =====
platform linux -- Python 3.11.15, pytest-9.0.2, pluggy-1.6.0
rootdir: /home/nathancharlesjones/Documents/Work/DigiKey/2026/articles/howHardwareGetsHacked_Part2/code/testing
configfile: pytest.ini
collected 15 items / 14 deselected / 1 selected

testing/test_security.py F

===== FAILURES =====
testing/test_security.py:297: in test_birthday_bound_attack_quick_check
    assert projected_attack_s >= CONCERNING_THRESHOLD_S, (
E   AssertionError: A birthday-bound table/oracle attack against this 32-bit nonce could plausibly succeed in as little as ~435.7s (~0.01 days)
pts at the protocol's own wire rate - too low to be comfortable. Pass --run-oracle-attack-full to reproduce this directly.
E   assert 435.74058226635776 >= 3600
----- Captured stdout call -----

Observed nonce width: 32 bits (4,294,967,296 possible values)
Observed wire time per unlock: 3.993 ms
Projected birthday-bound table attack (~50% odds): ~54,562 unlocks each way, ~435.7s (~0.01 days)
```

Other defenses

Widening the nonces may have been the obvious solution, but there are others that we could consider. For instance, we could apply to this situation the same reasoning that was described in [Part 8](#) about protecting the pairing pin from brute force attacks by:

- **Making each attempt categorically more difficult:** Requiring an attacker to provide a fingerprint scan alongside the unlock request or using a non-repeating PRNG algorithm and locking/erasing the device after 2^{32} total unlock attempts over the lifetime of the car.
- **Making each individual attempt take longer:** Enforce a rate-limit of one unlock per second or having a 1 minute timeout after 3 unsuccessful unlock attempts.
- **Decreasing the odds of any single guess being successful** by widening the nonce (which we did above)

As with the pairing pin, though, the competition and it's specific threat model eliminate many of these as viable options.

- We can't change the competition rules (or the hardware!) to require a fingerprint scan or other additional requirement to make unlocking more difficult

- Attackers can still reflash the hardware at will, making it meaningless to save counters or timeout values or to even have any sort of saved system state whatsoever (attackers in the competition could simply reflash the target with fresh firmware whenever they wanted, resetting any variables that were intended to be stored between power cycles)

Adding a roughly 1 second delay for each unlock request (the maximum time an unlock could take, per the competition rules) would definitely help, but it would be insufficient on its own to prevent the “birthday problem” attack.

/*****[? What about that “non-repeating PRNG algorithm” thing?]***/

The problem with pseudo-random number generation is that *every* possible answer should have the same probability of showing up on any given step, giving rise to the “birthday problem.” An alternative to this form of random number generation might be to encrypt a monotonically increasing counter using something like AES, a technique that was brought up during our initial discussion about PRNGs in [Part 7](#):

```
static uint32_t counter = 0;
uint32_t rand(void)
{
    // return upper 2 bytes of AES_CMAC(prng_key, counter++);
}
```

The counter would increment on each unlock, ensuring that the output from AES (the random number) would be different on each unlock and, critically, *non-repeating*⁴. (At least until the counter rolled over from $2^{32}-1$ to 0, at which point the car could just permanently disable itself, preventing any further unlocks. This number could be made high enough to never be reached during normal operation.)

This would seem to solve our “birthday problem” attack, since there’s no birthday problem to contend with! Unfortunately, it wouldn’t survive the attackers in this competition. Can you explain why? (Answerⁱⁱⁱ)

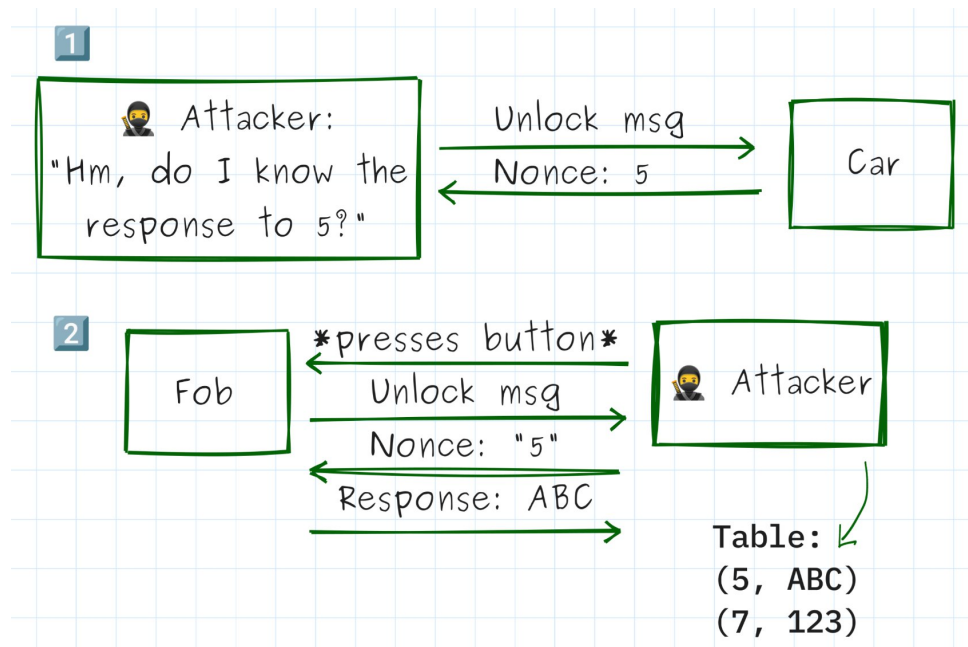
*****/

⁴ This non-repeating quality is also what makes this a **bad** PRNG construction, actually, since as numbers are produced by the algorithm it gives the remaining numbers a *higher* probability of being the next one produced. This is why the CTR-DRBG construction recommended by NIST includes several more steps than merely “encrypting an increasing counter”.

The problem with “oracles”

Part of the reason the “birthday problem” attack works at all is that both the fob and the car act as **oracles**: devices that an attacker can query at will to learn some information about the system.

1. Attackers can send unlock messages to a car whenever they want to see which nonce message is sent back.
2. They can also trigger an unlock on the fob and then send the fob a made up nonce message to see what response the fob sends back.

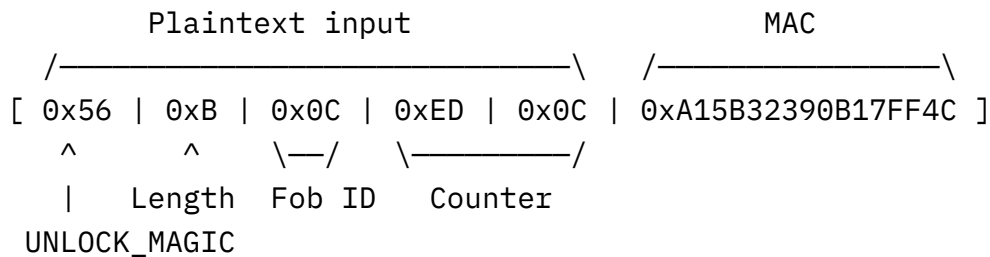


Right now, the “birthday problem” attack uses primarily the second fact above to generate its table of (nonce, response) pairs (though an attacker could also get that information by just observing valid unlock transactions, as was depicted originally) and then the first fact above during the actual attack to start an unlock sequence without the fob being present.

Ideally, neither the fob nor the car ever talks to a device without authenticating it as a valid car or fob, though. If the car had a way to verify that an unlock request came from an authentic fob, then it would cease to be an oracle, eliminating situation #1. And if the fob had a way to verify that the nonce messages it sees after sending an unlock request came from an authentic car, then it would cease to be an oracle in the sense of situation #2. Eliminating both the fob and car as oracles would *categorically* eliminate the “birthday problem” attack, not simply make it less likely to succeed.

I spent some time trying to think of ways to do that, however, and couldn't find a solution that still offered any meaningful improvements after accounting for the fact that attackers in the competition could reflash a car or fob whenever they wanted.

For example, say we try to eliminate the car as an oracle by expanding the unlock message (which is back to being [0x56 | 0x6 | 'unlock'] right now) to include a rolling counter and MAC value.



The car would refuse to send a nonce for any unlock message that didn't have the right counter and MAC values.

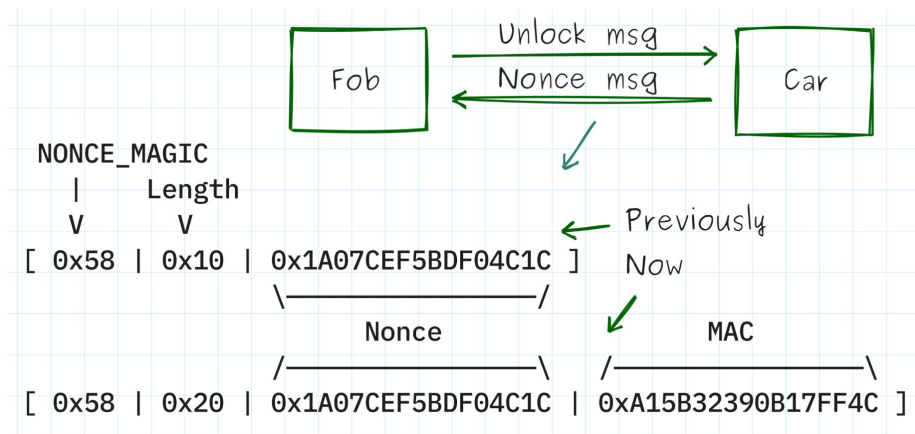
/*****[? Can you spot the problem?]***/

There's a problem with the above design! What is it? (Answer^{iv})

*****/

Fundamentally, systems that try to authenticate devices with a single message (static passwords, rolling codes + MAC, etc) end up being vulnerable to attacks that simply reflash the device and reset whatever state was supposed to be saved between unlock attempts. The challenge-response we developed in Part 7 fixed this, but at the cost of the car acting like an oracle, for which there isn't a workaround: the car can't withhold a challenge before a device has authenticated itself if it's using that challenge to authenticate the other device!

Or perhaps we try to eliminate the fob as an oracle by requiring that nonce messages also be accompanied with a MAC value (using a new, unique key, separate from the unlock key, of course!), verifying that they came from a real car.



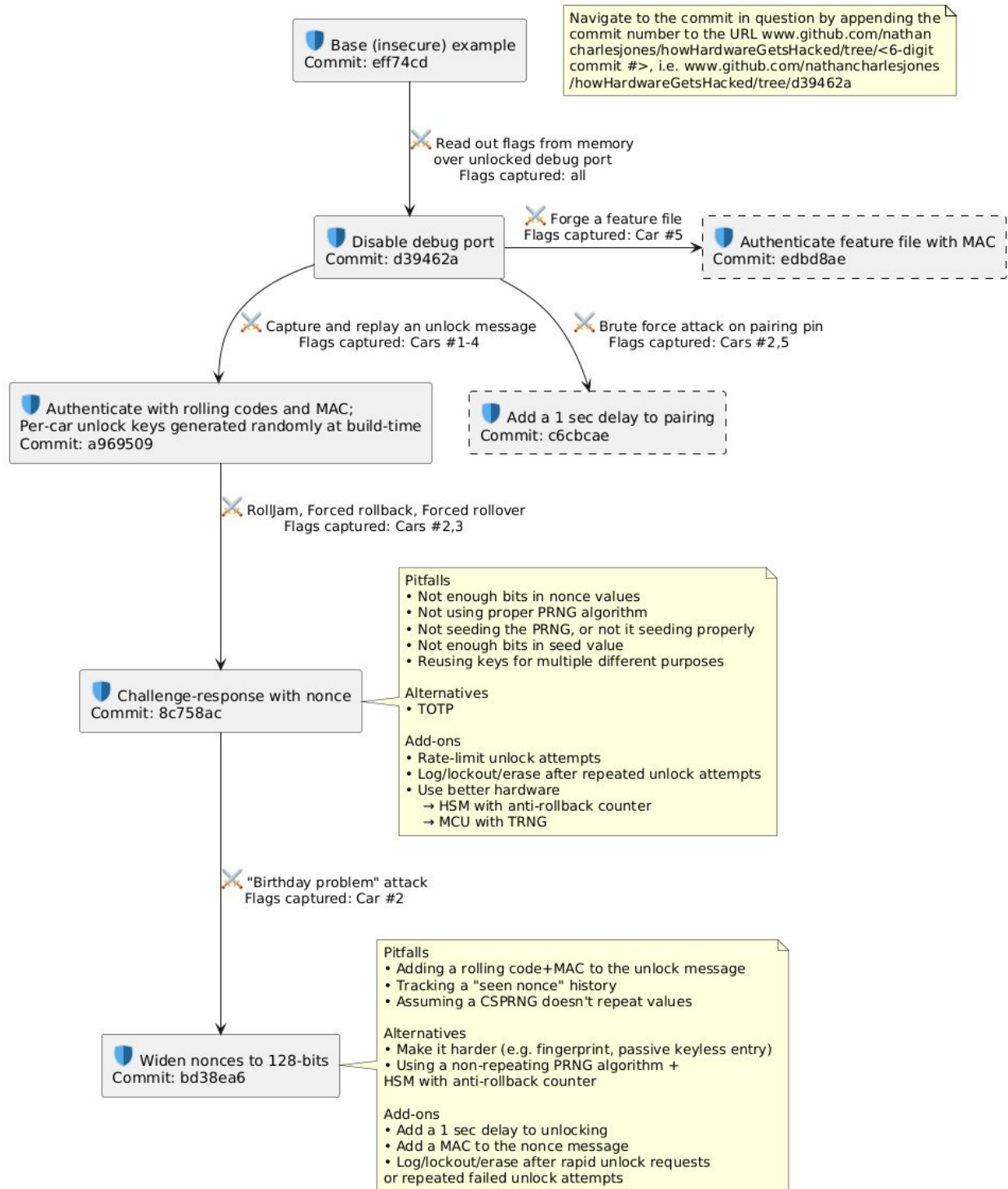
The fob would refuse to reply with its own MAC response (computed using the unlock key) unless it could validate the nonce MAC using its own “nonce key.”

This one is interesting, since it *does* prevent an attacker from querying a fob by itself to generate the table of (nonce, response) pairs. However, in both of the realistic attack scenarios that were mentioned above (a valet or an attacker who has installed a listening device outside a person’s home), this would have no effect on the attacker, since the fob is already interacting with an authentic car and the attacker is merely listening to that conversation.

All of which goes to show the importance of things like hardware monotonic counters (which *can’t* be reset to 0) and of designing your system so that an attacker can’t reset or reflash your device.

Updated threat model

Our updated threat model with this new attack (scoped just to show the relevant features) is below.



The primary **pitfall** (a plausibly wrong solution) in trying to defend against the “birthday problem” attack is thinking we could add a rolling code+MAC to the unlock message to close off the car as an oracle. This doesn’t work because it’s vulnerable to exactly the attacks we developed in [Part 7](#): RollJam, Forced rollback, and Forced rollover.

Two additional pitfalls are the ideas that:

- A valid PRNG algorithm like the one we're using *won't* repeat any values (they will; that's kind of the whole point of this article) and
- That it's possible to prevent a nonce from being repeated by simply tracking in software which ones have already been sent out. This pitfall is vulnerable to devices being reflashed and having that history reset.

The only two **alternatives** to widening the nonce (defenses that would or could have worked as well it) would have been to have:

- Added something to the unlock process like a fingerprint scan or requiring the fob to be within a certain distance of the car (such as in a **passive keyless entry** system, which uses RF signals to detect if a fob is near the car) or
- Replaced our PRNG with a non-repeating algorithm whose counter was stored in an anti-rollback counter.

Adding some sort of timeout after N unlock attempts within a certain small window might also have worked, had our threat model not included reflashing (i.e. had our design/the competition have prevented that from happening).

In addition to widening the nonce, there are a few things that could be **add-ons** to the design for additional security or better features.

- Adding a 0.75 or 1 sec delay to each unlocking (the maximum length of time the rules say unlocking is allowed to take) would make the “birthday problem” attack take a few hundred times longer than it would otherwise.
- Adding a MAC to the nonce message would prevent an attacker from querying a fob directly for response values, forcing them to obtain that data via a live unlock session between a valid fob and car.
- Had the hardware/rules allowed it, logging, locking, or erasing the device upon a rapid sequence of unlock requests or failed attempts could have made this attack much harder or even categorically infeasible.

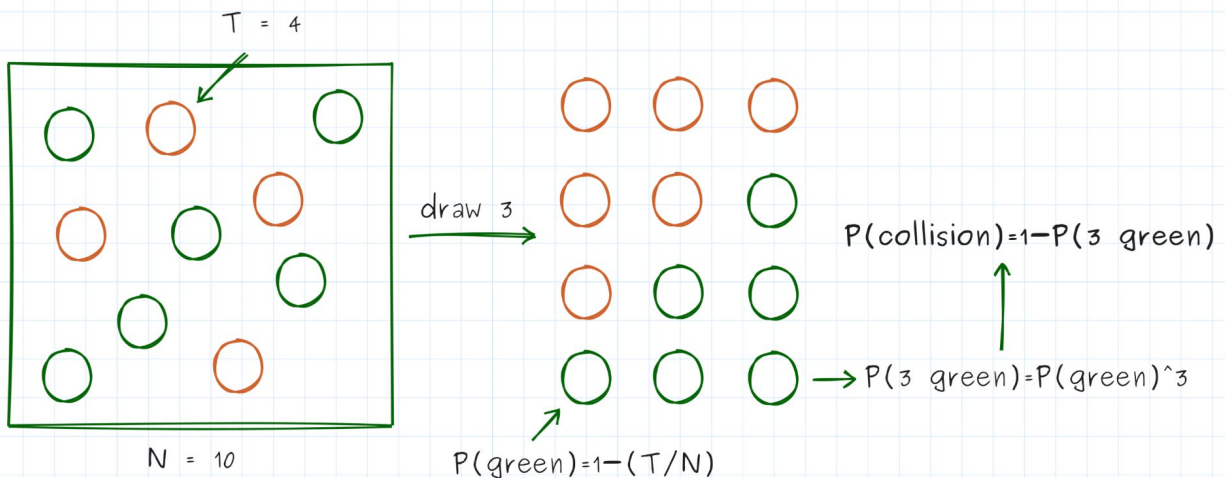
Conclusion

In this article we discovered that 4.3 billion isn't actually that big of a number and there's a real chance that a random number generated from that pool will repeat often enough to facilitate another replay attack! The simple solution was to generate nonces from a much larger pool (up to 2^{128} , instead of 2^{32}), making repeated values *much* less likely to occur.

Along the way, we discussed the idea of an “oracle”, including what it is and why it can’t always be avoided.

If you’ve made it this far, thanks for reading and happy hacking!

ⁱ The probability works like this: pretend you have a box of 10 green balls (N) and you pull out 4 of them (T) and paint them orange. If you reach in to pull out a ball 3 times (M), what’s the probability of pulling out at least one orange ball? Well, there are only four possible outcomes in terms of the colors of those 3 balls: 3 orange, 2 orange/1 green, 1 orange/2 green, or 3 green. The probability of drawing at least one orange ball (the “birthday problem”) is the summed probability of the first three scenarios or, put another way, “1 – the probability of the last scenario”. The probability of the last scenario is the chance of drawing a green ball, $\frac{N-T}{N}$ or simply $1 - \frac{T}{N}$, M times in a row, which has a probability of $(1 - T/N)^M$. In this case, that’s $P(\text{collision}) = 1 - (1 - 4/10)^3 = 0.784$.



ⁱⁱ It seems reasonable to me to guess that a person might unlock their car 2-3 times a day, which we’ll approximate as 1000 total unlocks over the course of a year. Thus, an attacker would need $M = \frac{\log(1-P)}{\log(1-T/N)} = \frac{\log(1-0.5)}{\log(1-1000/4.3 \text{ billion})} = 2,977,044$ attempts using that table to have a 50% chance of unlocking the car, which they could accomplish in as little as 1.58 hours.

ⁱⁱⁱ The “non-repeating PRNG algorithm” wouldn’t survive the attackers in the competition since they can reflash the car at will, resetting the counter to 0. We defeated that system in [Part 7](#) by simply recording valid (nonce, response) pairs and then resetting the car, making all those responses valid again since the car was about to reissue all of the same nonces! If this were a production device and we could add a hardware security module (HSM) with an anti-rollback counter on it, however, this could be a viable defense.

^{iv} The fundamental problem is that this is exactly the design we arrived at in [Part 6](#) and subsequently broke in [Part 7](#)! Thus, “authenticating the fob using a rolling code + MAC” is defeated by any of the attacks we looked at in Part 7 (RollJam, Forced Rollback, Forced Rollover) and, plus, it makes the subsequent challenge-response rather redundant.